Choose Your AI Project

CS453 | Individual work | Two-week independent checkpoint

One choice now; more AI techniques later

Choose ONE project below. During the professor's two-week absence, complete the agent model, small simulation and taught search methods. After the professor returns, extend the same project as new topics are taught. The entire course project is not due in these two weeks.

Choice	Goal and task action
1. Campus Delivery Agent	Start with a parcel. Reach its target building and deliver it. State includes location and parcel status. Delivery is legal only at the assigned destination.
2. After-Hours Cleaning Robot	Reach one requested room and clean it. State includes location and whether that room is dirty. Cleaning a remote room is illegal. An already-clean task needs no cleaning action.
3. Library Retrieval Agent	Reach one shelf zone and retrieve one requested item. State includes location and item possession. Retrieval is legal only in the assigned zone.

Use a Python simulation, not real hardware. All choices have comparable scope and the same evidence requirements. Several students may choose the same option, but everyone submits their own implementation, outputs and explanation. Use the workbook's repeatable map/target variant rule.

What is required during these two weeks

Use **BFS (uninformed) in Task 5 and A* (informed) in Task 6**. The workbook supplies admissible heuristic tables. All roads have unit cost in this checkpoint. Show each assigned route, explain the method and simulate the final task action. Separate test tables, results and limitations sections are removed. UCS, inference engines, ML, websites and physical robots are not required here.

Time and submission

Plan approximately **8-12 hours over two weeks**. Submit the completed work at the end-of-absence deadline shown in Classroom. Feedback and the individual explanation take place after the professor returns. No new approval or external dataset is needed during the absence.

Your supplied world and schedule

Reference graph: eight locations, undirected unit-cost roads

Locations: **S, A, B, C, D, E, F, G**. Roads: **S-A, S-B, A-C, A-D, B-E, C-F, D-G, E-G, F-G**. Every road works in both directions and every move costs 1. The workbook supplies this data and the variant helper; you implement the agent and search logic.

The graph and variant helper are supplied data. Task 3 (world and hand trace) has been removed; you do not need a separate diagram or hand trace. Keep the generated variant in your notebook. Task numbering remains unchanged so the remaining sections match the original workbook.

Your personal variant

In the workbook, enter your student ID. Set v to the sum of its character codes modulo 4. The helper removes one road and assigns a target using the table. Start at S. Keep the printed variant in your submission. Students may receive matching variants; individual work is still required. Rename locations in your explanation to fit your chosen story.

v	Remove road	Target
0	A-D	G
1	B-E	F
2	C-F	E
3	E-G	D

Week 1: establish a working baseline

Days 1-2: choose your story; define PEAS and classify the initial environment. Write state, legal actions, transitions, goal and path cost. Generate your supplied variant.
Days 3-4: implement legal movement, the task action and state updates. Reject impossible actions clearly. Keep the simulator separate from search.
Days 5-7: implement BFS with a FIFO queue, repeated-state handling and route reconstruction. Show your assigned mission output. No interim submission or approval is required.

Week 2: compare and finish

Days 8-10: implement A* with a priority queue and g+h ordering, using the supplied heuristic table. Explain why the values are lower bounds on the assigned map.
Days 11-12: show A* executing the assigned mission; finish the method explanations and sources disclosure.
Days 13-14: complete README, verify clean execution, commit and submit. No separate tests/results/limitations sections or interim submission are required.

Acceptance criteria

A complete checkpoint contains the following evidence. Do not submit only screenshots or an unchanged workbook.

Task 1. Agent and environment

Record your identity, selected story and variant; define PEAS and justify the environment classification.

Task 2. Representation

Define state, legal actions, transitions, goal and travel cost. Include task status as well as location.

Task 4. Simulator

Implement legal movement and the delivery/cleaning/retrieval service at the target. Update task state correctly; handle already-completed tasks and invalid actions clearly.

Task 5. BFS - uninformed

Implement FIFO search with repeated-state tracking and route reconstruction. Explain why BFS is uninformed and returns minimum moves for unit costs. Show its assigned mission output.

Task 6. A* - informed

Implement a priority queue, g+h ordering, best-cost updates and path reconstruction. Test the goal on valid removal. Explain g, h and admissibility of the supplied table; show the assigned mission output.

Tasks 10-11. Sources and submission

Cite adaptations, disclose assistance and follow the GitHub/Colab submission checklist. Clean execution and individual explanation remain required.

Supplied heuristic and its scope

The heuristic tables give shortest distances in the original graph before the assigned road closure. Deleting roads cannot reduce shortest distance, so these estimates remain lower bounds. h(goal)=0. They are supplied data: students implement the actual BFS and A*. If shortcuts or cheaper edge costs are introduced, justify a new heuristic or use h=0.

Removed tasks

Tasks 3, 7, 8 and 9 are removed. There is no separate hand trace, ten-case test table, comparison/results section, limitations essay or Day 7 checkpoint. Original task numbers are retained. The implementations must still behave correctly on valid, unreachable and invalid inputs; this is part of writing the algorithms, not a separate test-table assignment.

Working independently

When you get stuck

1. Save the exact input, error and last working version.
2. Reduce the problem to two or three locations and trace the queue/priority queue by hand.
3. Consult the supplied graph-search and informed-search STUDENT notebooks. Distinguish exploration order from route reconstruction.
4. Fix one issue at a time. Record what you tried and the result.
5. Continue independent parts such as explanations and documentation. Keep a questions log for the professor's return. Do not depend on immediate replies during the absence.

Permitted help and individual ownership

You may use course code as a cited starting point and adapt it to your task. You must explain the adaptation and understand it. Do not share submitted code, analysis or outputs with classmates. AI may explain concepts, suggest debugging and review code under the course policy; disclose the tool, help and your verification. Do not upload private records or credentials.

Submit a GitHub repository

Keep **project.ipynb**, **README.md** and required files in your own private GitHub repository. Use Colab for coding and execution. Submit the **repository URL and final commit URL** in Classroom by its deadline, then Turn in. Invite the professor using the announced GitHub username. Pages 5-6 explain setup, saving and submission. No terminal or local Git installation is required.

How the project grows after this checkpoint

After the relevant lessons: extend cost-aware search with weighted roads and UCS; introduce rules and explanation traces; compare supplied route alternatives under uncertain delay; then fit one small classifier on supplied synthetic task records. End with fair evaluation, limitations and an individual demonstration. Later-stage criteria/materials will be released before they are assessed. Other topics such as minimax, neural networks and RL remain in regular labs.

Assessment

This checkpoint provides evidence for the final project; it is not an extra assessment on top of it. The course plan proposes **40 course points (40%)**; the professor confirms the approved weight. Later stages contribute to the same final rubric. Individual oral explanation follows after the professor returns.

Final project criterion	Points
Problem and plan / Working baseline	3 / 5
Improved AI method / Testing and comparison	10 / 7
Error analysis / Code and reproducibility	4 / 3
Demonstration / Individual oral explanation	3 / 5
TOTAL	40

Correct implementations and honest analysis can receive full credit even where a comparison method performs worse. Inability to explain code or make a small live change limits oral marks to 2/5 under the existing handbook. Administrative and integrity issues follow the published course/university policy.

GitHub + Colab: setup

1. Create a private repository

Sign in to GitHub, create **cs453-ai-project**, select **Private** and initialize a README. Keep the same repository for later stages. Under **Settings > Collaborators > Add people**, invite the professor using the username announced in Classroom. Begin work without waiting for acceptance; the invitation must be accepted before the professor can view private work. Do not invite classmates.

2. Add the workbook

Download the Classroom workbook and rename your copy **project.ipynb**. On GitHub, use **Add file > Upload files** and commit the upload as **Start project workbook**. Your required files are **README.md** and **project.ipynb** at the repository root. The initial graph needs no data folder.

3. Open the notebook in Colab

Visit colab.research.google.com/github and find your repository/notebook. For a private repository, enable **Include Private Repos** and authorize access if prompted. A notebook opened in Colab is an editable view; saving to Drive alone does **not** update GitHub.

4. Explicitly save progress to GitHub

In Colab, use **File > Save a copy in/to GitHub**. Select your own repository, the correct branch (normally **main**) and the existing path **project.ipynb**. Add a meaningful commit message. Authorize if needed, then check the file and latest commit on GitHub. Use one current working copy and avoid duplicate filenames.

If account linking or saving fails

Download the notebook from Colab as .ipynb. Name it **project.ipynb**, then upload it through GitHub to replace that same file and commit. To resume, download the latest GitHub version and upload it into Colab. This browser-only fallback does not require connecting accounts. Never paste GitHub passwords or access tokens into notebook cells.

Keep meaningful progress

Commit setup, agent/simulator work, BFS, A* and final review. Messages should describe real changes, such as **Add BFS baseline**. History supports your explanation; the number of commits is not a separate marking target. Do not fabricate history.

GitHub + Colab: submit

5. Complete README.md

Use the supplied repository README template. Include name/ID, GitHub username, project choice, printed variant, files, run steps, progress and algorithm explanations. Cite adapted course code and disclose AI help. Keep the assessment repository private.

Add an Open in Colab link pointing to your actual username, repo, branch and **project.ipynb**. The companion instructions and README template give the exact Markdown. Private-repository links still require permission/authorization. Opening a notebook does not load other repository files into its runtime: document loading steps if you add files later.

6. Check the version that will be marked

A. Restart the Colab session, run all cells and inspect assigned mission outputs.
B. Commit the finished notebook and README to GitHub; verify outputs are present.
C. Reopen/download that committed notebook into a fresh Colab session and run it. If you change content, recommit and use the new final commit.
D. Open the final commit details on GitHub and copy its URL, containing **/commit/** and the full commit ID.

7. Turn in through Google Classroom

Attach your **repository URL** and **final commit URL** to the existing assignment using **Add or create > Link**, then select **Turn in**. If only one link is accepted, attach the repository and place the final commit URL in a private assignment comment. State your chosen project and GitHub username. The professor must have private-repository access; sending a link alone is insufficient.

The commit identifies the submitted checkpoint even when you continue the project later. Follow the announced deadline and late policy for any revised submission. Send the professor invitation at setup; you can work while it is pending. Do not make the repository public to resolve an access problem.

Final GitHub checklist

My private repo contains README.md and project.ipynb; my variant and help disclosure are complete; meaningful progress is saved in commits; the saved notebook runs from a fresh Colab session; the professor invitation is sent; Classroom contains both repository and final commit links; I selected Turn in.

Official interface references

Google Colab: GitHub integration
GitHub: create a repository
GitHub: invite collaborators
GitHub: upload files